Praktikum Pemrograman Berorientasi Objek

Pertemuan 8

Materi Pertemuan 8

Gambaran Materi Pembelajaran Hari Ini

Materi Pertemuan 8

01

JDBC

Menghubungkan kode Java dengan database relasional melalui API java.sql

02

Connection Pool

Strategi pemakaian ulang koneksi database untuk performa dan efisiensi sumber daya

Pengenalan JDBC

Definisi

Java Database Connectivity (JDBC) adalah API standar Java (paket `java.sql`) yang menghubungkan kode OOP dengan *relational database* melalui perintah **SQL**.

Peran JDBC

JDBC menyediakan satu antarmuka seragam, sehingga kode program tidak perlu mengetahui detail tiap DBMS (MySQL, PostgreSQL, Oracle).

Agar dapat terhubung, diperlukan **JDBC driver** sesuai DBMS yang dipakai, umumnya berbentuk file `.jar`.

Komponen Inti:

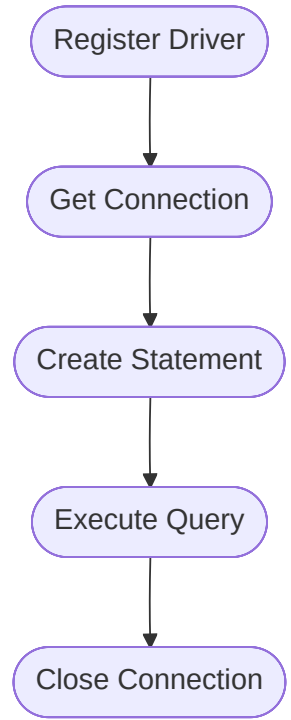
- Driver Manager
- Connection
- PreparedStatement
- ResultSet

Komponen Inti JDBC

Komponen	Peran
Driver Manager	Mengelola daftar driver database dan membuka koneksi baru. Berbentuk file `.jar`
Connection	Mewakili sesi transaksi aktif dengan server *database*
PreparedStatement	Mengirim instruksi *query* SQL ke database
ResultSet	Menampung dan membaca tabel data hasil eksekusi *query*

Alur Eksekusi JDBC

1. **Register Driver** – Menyiapkan driver DBMS
2. **Get Connection** – Membuka koneksi ke *database*
3. **Create Statement** – Membuat instruksi SQL
4. **Execute Query** – Menjalankan *query* & mengambil hasil
5. **Close Connection** – Menutup koneksi



Implementasi Koneksi Database

Class DatabaseConnection bertanggung jawab membuka dan mengelola satu koneksi ke database.

- `URL`, `USER`, `PASS` disimpan sebagai konstanta.
- Koneksi dibuat sekali, lalu dipakai ulang selama belum tertutup.
- `DriverManager.getConnection()` membuka koneksi fisik.

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/database";
    private static final String USER = "root";
    private static final String PASS = "";

    private static Connection conn;

    public static Connection getConnection() {
        try {
            if (conn == null || conn.isClosed()) {
                conn = DriverManager.getConnection(URL, USER, PASS);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return conn;
    }
}
```

Model `User`

Representasi tabel users sebagai object Java.

Atribut class menyesuaikan kolom pada tabel, dan `idUser` diisi setelah data berhasil disimpan ke database.

```
import java.sql.*;

public class User {

    private int idUser;
    private String username;
    private String password;
    private String email;
    private String role;

    public User(String username, String password, String email, String role) {
        this.username = username;
        this.password = password;
        this.email = email;
        this.role = role;
    }
}
```

`User`: Menyimpan Data (Insert)

Mengirim perintah INSERT dengan PreparedStatement.

Tanda `?` adalah *placeholder* yang diisi lewat `setString()`, sehingga terhindar dari SQL Injection.

```
    public int registerUser(Connection conn) throws SQLException {
        // Menerima koneksi dari luar (Transactional)
        String sql = "INSERT INTO users (username, password, email, role) VALUES (?, ?, ?, ?)";
        try (PreparedStatement pst = conn.prepareStatement(sql)) {
            pst.setString(1, this.username);
            pst.setString(2, this.password);
            pst.setString(3, this.email);
            pst.setString(4, this.role);
            pst.executeUpdate();

            ResultSet rs = pst.getGeneratedKeys();
            if (rs.next()) {
                // Simpan ID yang baru dibuat ke object ini juga
                this.idUser = rs.getInt(1);
                return this.idUser;
            }
        }
        return 0;
    }
```

`User`: Membaca Data (Select)

Menjalankan SELECT dan membaca hasilnya dari ResultSet.

`rs.next()` memindahkan kursor ke baris berikutnya; bila tidak ada baris, method mengembalikan `null`.

```
    public String getUsernameById(Connection conn, int idUser) throws SQLException {
        String sql = "SELECT username FROM users WHERE id_user = ?";

        try (PreparedStatement pst = conn.prepareStatement(sql)) {
            pst.setInt(1, idUser);

            try (ResultSet rs = pst.executeQuery()) {
                if (rs.next()) {
                    // Mengembalikan username yang ditemukan
                    return rs.getString("username");
                }
            }
        }
        // Mengembalikan null jika id_user tidak ditemukan
        return null;
    }
```

Menjalankan Program (`Main`)

Program menyimpan user baru, lalu membaca kembali username berdasarkan ID-nya.

```
import java.sql.Connection;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        try (Connection conn = DatabaseConnection.getConnection()) {
            User user = new User("banung", "ayojadiasprak", "banung@gmail.com", "user");

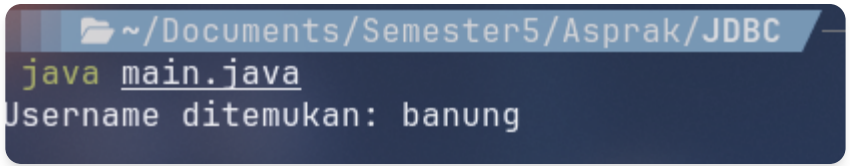
            user.registerUser(conn);

            // Panggil method untuk mendapatkan username
            String username = user.getUsernameById(conn, 0);

            if (username != null) {
                System.out.println("Username ditemukan: " + username);
            } else {
                System.out.println("User dengan ID tersebut tidak ditemukan.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Hasil Implementasi

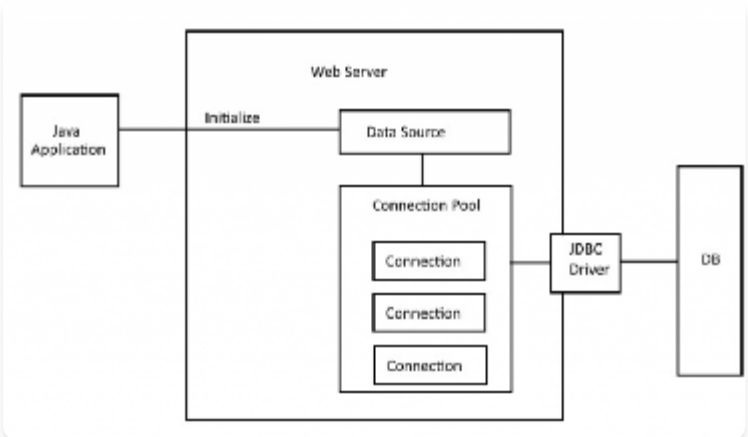
Menjalankan Main akan menyimpan data user ke database, lalu menampilkan username yang berhasil dibaca kembali.



Connection Pool

Manajemen Koneksi Database yang Efisien

Connection Pool



Definisi

Teknik manajemen koneksi database yang membuat dan menyimpan sejumlah objek koneksi (*pool*) secara *pre-allocated* saat aplikasi menyala.

Cara Kerja

Aplikasi meminjam koneksi yang sudah siap di dalam *pool* saat query, lalu mengembalikannya kembali ke *pool* (tanpa menutup koneksi fisik).

Manfaat

Eliminasi *overhead* otentikasi jaringan, menekan latensi query, dan mencegah database *overload*.

Library

HikariCP (*default* di Spring Boot), Apache DBCP, C3P0.

Sumber: https://www.digitalocean.com/community/tutorials/connection-pooling-in-java

Ringkasan Connection Pool

Connection Pool adalah strategi pemakaian ulang (*reuse*) koneksi database yang siap pakai untuk meningkatkan performa dan mencegah server kehabisan sumber daya.

Tanpa Pool

Setiap query membuka dan menutup koneksi baru.

Dengan Pool

Koneksi dipinjam dari *pool*, lalu dikembalikan.

Hasil

Latensi turun dan database terhindar dari *overload*.

Latihan Minggu 8

Kerjakan latihan sebagai bagian dari pendalaman materi!

https://drive.google.com/drive/folders/1SOgPinXEaQuLWWtDFQ_XoR7_-FNzlQLh?usp=drive_link

Instruksi Latihan

Lanjutkan proyek *Course Registration* minggu 7 (seluruh TODO lama sudah selesai).

Sekarang persistensikan data ke **database** melalui **JDBC**. Implementasikan ketiga *repository* di dalam `repository/jdbc/`.

- `JdbcStudentRepository` — `save`, `findById`
- `JdbcCourseRepository` — `save`, `findById`
- `JdbcEnrollmentRepository` — `findActiveByStudentAndCourse`, `countActiveByCourseId`, `save`, `update`

- Cari komentar `TODO` beserta `HINT` pada tiap method.
- Jalankan `mvn test` — 6 test JDBC (H2 in-memory) harus lulus.

Terima Kasih!

Semangat UTS-nya :)